

---

# Documentação de Especificação e Desenvolvimento de Software

**Projeto:** Sistema de Gestão e Vitrine Web - Rauber Festas e Eventos

**Responsável:** Luis Henrique Rauber

**Cliente:** Rauber Festas e Eventos - Roberto Egídio Rauber

---

## 1. Visão Geral do Projeto

O projeto consiste no desenvolvimento de uma aplicação web híbrida para o espaço "Rauber Festas e Eventos". O sistema atua em duas frentes estratégicas:

1. **Vitrine Pública (Front-office):** Divulgação do espaço, pacotes e consulta de disponibilidade de datas em tempo real (com privacidade dos dados sensíveis).
2. **Sistema de Gestão (Back-office):** Painel administrativo para controle total de cadastro de eventos, gestão financeira (status de pagamento visual) e histórico de locações.

## 2. Atores do Sistema

- **Visitante (Cliente):** Usuário externo que acessa o site para visualizar o espaço, conferir preços e verificar datas livres no calendário.
- **Administrador (Gestor):** Usuário interno (proprietário) responsável pela gestão da agenda, upload de fotos da galeria e controle financeiro dos eventos.

## 3. Requisitos Funcionais (RF)

### Módulo 1: Vitrine Virtual (Área do Cliente)

- **[RF001] Visualizar Informações e Galeria:** O sistema deve exibir fotos reais do espaço, categorizadas (Casamento, Infantil, etc.) e gerenciáveis pelo administrador via upload.

- **[RF002] Visualizar Planos de Preços:** O sistema deve exibir três modalidades claras de contratação:
  - a. Segunda a Quinta: R\$ 1.500,00.
  - b. Fim de Semana e Feriados: R\$ 2.000,00.
  - c. Corporativo/Treinamentos: Valor sob consulta.
- **[RF003] Mapa de Localização:** Integração via iframe com Google Maps mostrando o endereço exato.
- **[RF004] Calendário de Disponibilidade Pública:** O sistema deve exibir um calendário interativo onde:
  - a. Dias Livres: Visíveis/brancos.
  - b. Dias Ocupados: Marcados em vermelho ("Indisponível"), sem exibir detalhes sensíveis do cliente (Visualização Sanitizada).
  - c. Dias Passados: Bloqueados visualmente.
- **[RF005] Acesso Administrativo Rápido:** O sistema deve conter um link discreto "Área Restrita" no rodapé para facilitar o acesso do gestor.

## **Módulo 2: Painel Administrativo (Área do Gestor)**

- **[RF006] Autenticação:** Acesso restrito via email e senha criptografada ou segura.
- **[RF007] Upload de Fotos:** O administrador deve poder enviar fotos do computador para a galeria do site através de uma interface de upload.
- **[RF008] Cadastro de Reservas:** Permite registrar manualmente um evento com: Nome, Data, Status Financeiro e Campo de Observações Livre.
- **[RF009] Visualização de Agenda (Lista):**
  - a. Listagem dos Próximos Eventos ativos.
  - b. Ordenação Cronológica Crescente (do evento mais próximo para o mais distante).
  - c. Indicador visual de pagamento (Vermelho: Não Pago, Amarelo: Parcial, Verde: Pago).
- **[RF010] Visualização de Agenda (Calendário):** Aba "Calendário" exclusiva do admin que exibe todos os eventos do mês. Ao clicar, exibe detalhes completos (Cliente, Valor, Obs).
- **[RF011] Edição de Evento:** Funcionalidade para alterar dados de um evento já cadastrado (ex: cliente pagou o restante).
- **[RF012] Finalização de Evento:** Botão "Finalizar" que altera o status do evento para "CONCLUÍDO", removendo-o da lista de próximos e movendo para o histórico.
- **[RF013] Histórico de Eventos:** Área específica para consulta de eventos passados.

## **4. Requisitos Não Funcionais (RNF)**

- [RNF001] **Interface:** Utilização do framework **Bootstrap 5** para responsividade total (Mobile First), garantindo acesso via Desktop e Celular.
- [RNF002] **Biblioteca de Calendário:** Utilização da biblioteca **FullCalendar** (JavaScript) para renderização das agendas.
- [RNF003] **Armazenamento de Imagens:** Utilização da biblioteca **multer** (Node.js) para upload e armazenamento local de arquivos de mídia.
- [RNF004] **Banco de Dados:** Persistência de dados utilizando SGBD **MySQL**.

## 5. Regras de Negócio (RN)

- [RN001] **Privacidade da Agenda:** O calendário público **jamais** deve expor dados sensíveis (nome, telefone, valor) dos clientes agendados. A API pública deve retornar apenas as datas ocupadas.
  - [RN002] **Bloqueio Retroativo:** O sistema deve tratar dias passados como indisponíveis visualmente.
  - [RN003] **Ciclo de Vida do Evento:** Um evento ativo só pode ser movido para o Histórico mediante ação manual de "Finalizar" pelo administrador.
- 

## 6. Metodologia de Desenvolvimento (Modelo em Espiral)

O projeto seguiu o modelo iterativo em Espiral, conforme solicitado, dividido em 4 ciclos principais:

### Iteração 1 (Prototipação Visual):

- Definição dos requisitos com o cliente.
- Desenvolvimento do Front-end estático (`index.html`) para validação visual da vitrine e pacotes.
- *Resultado:* Aprovação da identidade visual e layout.

### Iteração 2 (Estrutura de Dados e Back-end):

- Modelagem do Banco de Dados MySQL (`rauber_db`).
- Configuração do servidor Node.js e criação das APIs básicas (Rotas GET/POST).
- *Resultado:* Sistema capaz de armazenar e ler dados.

### Iteração 3 (Funcionalidades Administrativas):

- Implementação do Painel Administrativo (admin.html).
- Desenvolvimento do sistema de Upload de Fotos (multer) e Gestão de Cores de Pagamento.
- *Resultado:* Cliente capaz de gerenciar o conteúdo.

#### Iteração 4 (Refinamento e Calendário):

- Integração da biblioteca FullCalendar (Público e Privado).
- Implementação da lógica de "Finalizar Evento" e ordenação cronológica.
- Testes finais de responsividade e segurança de rotas.
- *Resultado:* Sistema completo e funcional.

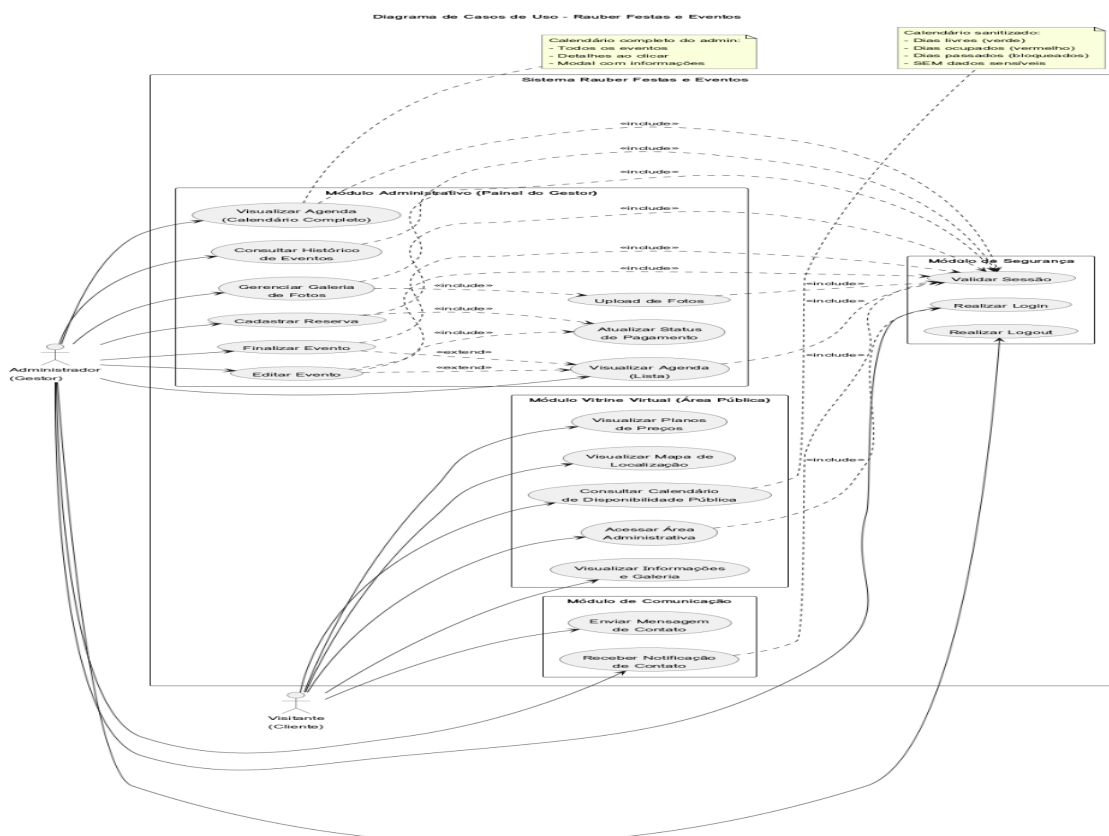
## 7. Tecnologias Utilizadas

- **Front-end:** HTML5, CSS3, JavaScript (ES6+), Bootstrap 5, FullCalendar.
- **Back-end:** Node.js, Express.
- **Banco de Dados:** MySQL 8.0.
- **Ferramentas:** MySQL Workbench, VS Code, Git e GitHub.

## 8. Estrutura de Diagramas

### 1. Diagrama de Casos de Uso

O Diagrama de Casos de Uso oferece uma visão de alto nível das funcionalidades do sistema e de como os atores (usuários) interagem com ele. Ele é fundamental para definir o escopo do projeto e garantir que todos os requisitos do cliente sejam atendidos.



## Descrição Detalhada

O diagrama identifica dois atores principais:

- Visitante (Cliente): Representa o usuário externo que acessa a parte pública do site. Suas principais interações incluem visualizar informações, a galeria de fotos, os planos de preços, o mapa de localização e, crucialmente, consultar o calendário de disponibilidade pública. Este ator também pode iniciar o contato com a empresa através do envio de uma mensagem.

- Administrador (Gestor): Representa o usuário interno com privilégios elevados. Este ator é responsável por toda a gestão do sistema, incluindo o gerenciamento da galeria de fotos, o cadastro e a gestão completa dos eventos (reservas), a visualização da agenda em múltiplos formatos (lista e calendário detalhado) e a consulta ao histórico de eventos. Todas as funcionalidades administrativas exigem autenticação prévia, representada pela relação de <<include>> com o caso de uso "Validar Sessão".

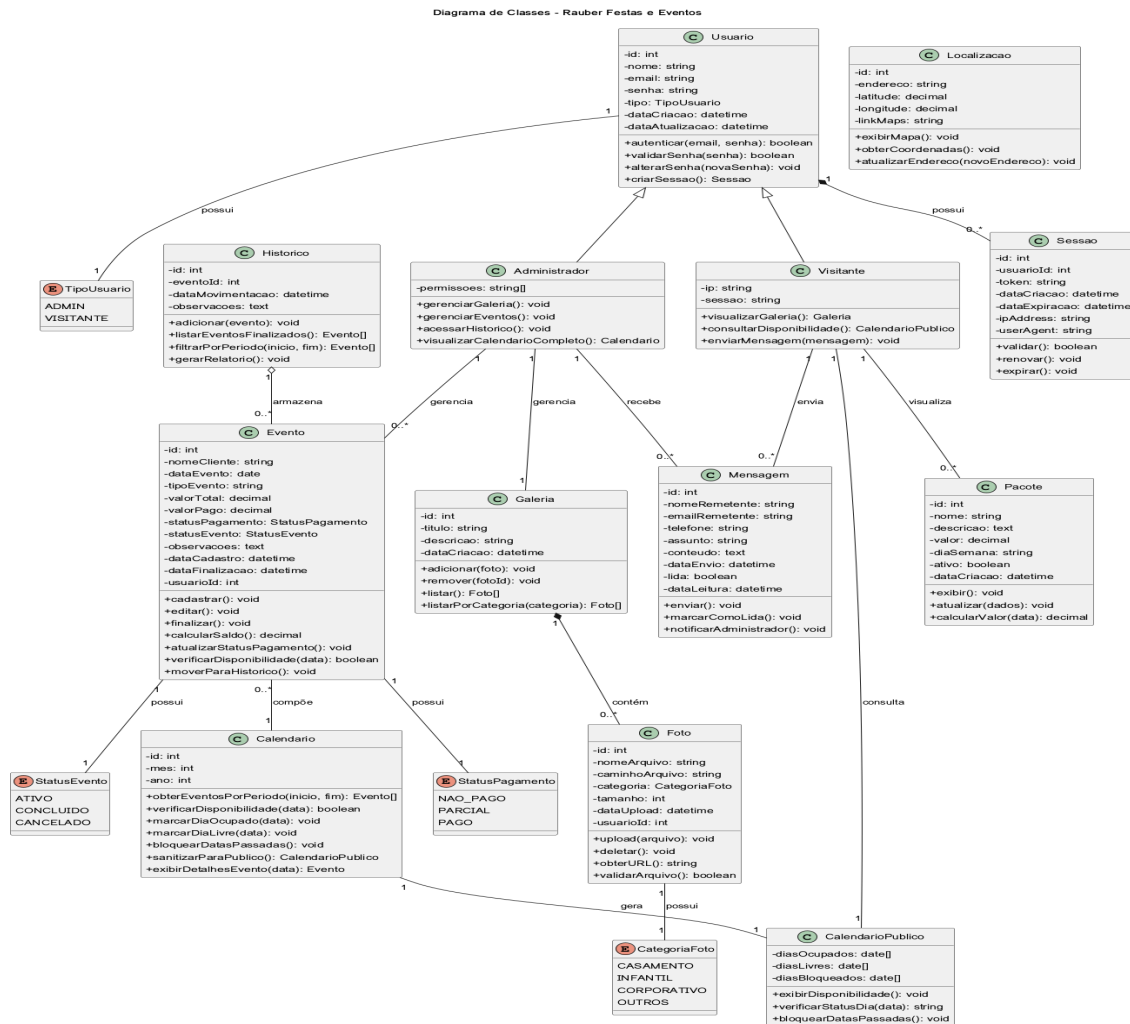
Os casos de uso estão organizados em quatro pacotes lógicos que representam os módulos do sistema:

- 1.Módulo Vitrine Virtual (Área Pública): Agrupa as funcionalidades disponíveis para o visitante.
- 2.Módulo de Segurança: Centraliza as funcionalidades de autenticação e autorização, como login, logout e validação de sessão.
- 3.Módulo Administrativo (Painel do Gestor): Contém todas as funcionalidades restritas ao administrador para a gestão do negócio.
- 4.Módulo de Comunicação: Gerencia a troca de mensagens entre os visitantes e o administrador.

As notas no diagrama destacam a diferença crucial entre o Calendário de Disponibilidade Pública (sanitizado, sem dados sensíveis) e o Calendário Completo do administrador (com todos os detalhes do evento), um requisito chave do projeto.

## 2. Diagrama de Classes

O Diagrama de Classes detalha a estrutura estática do sistema, descrevendo as classes, seus atributos, métodos e os relacionamentos entre elas. Ele serve como um blueprint para o desenvolvimento do código e do banco de dados.



3.

## Descrição Detalhada

O diagrama apresenta as principais classes que compõem o sistema:

- Usuario:** Uma classe base que define atributos e comportamentos comuns a todos os usuários, da qual herdam Administrador e Visitante.
- Evento:** Classe central que representa uma reserva. Contém todos os dados da locação, como nomeCliente, dataEvento, valorTotal, valorPago, e os status de pagamento e do evento, que são controlados por enum.

- Calendario:** Responsável por gerenciar a disponibilidade de datas. Possui um método fundamental, `sanitizarParaPublico()`, que gera uma instância de `CalendarioPublico`, garantindo a privacidade dos dados dos clientes (RN001).

- CalendarioPublico:** Uma versão simplificada do calendário, contendo apenas listas de dias ocupados, livres e bloqueados, sem nenhuma informação sensível.

- Galeria e Foto:** A classe `Galeria` gerencia um conjunto de objetos `Foto` através de uma relação de composição. A classe `Foto` armazena metadados da imagem, como caminho do arquivo e categoria.

- Sessao:** Gerencia a autenticação do usuário, armazenando o token JWT e controlando sua validade.

- Historico:** Armazena eventos que já foram finalizados, mantendo um registro de longo prazo.

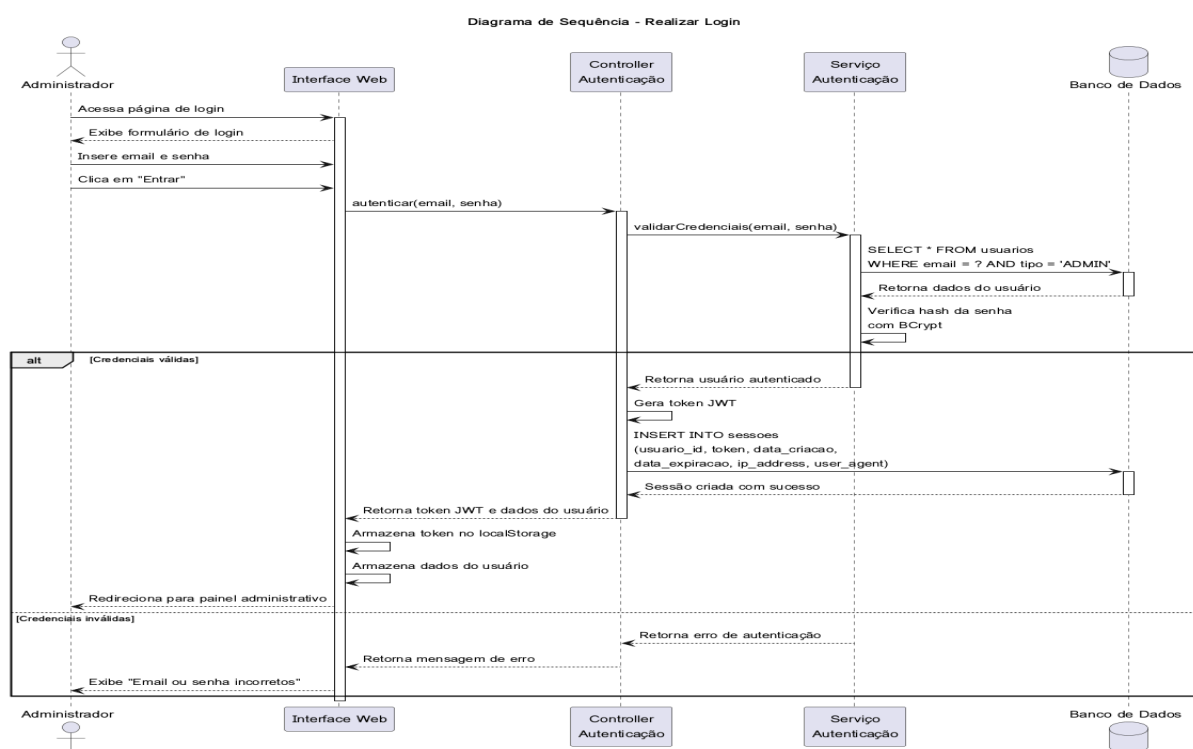
- Mensagem, Pacote, Localizacao:** Classes que modelam as demais entidades importantes do sistema.

Os relacionamentos de herança, composição, agregação e associação são claramente definidos, mostrando como as classes interagem. O uso de enum para status e tipos (como `StatusPagamento`, `StatusEvento`, `TipoUsuario`) garante a consistência dos dados em todo o sistema.

### 3. Diagrama de Sequência

#### 3.1. Sequência de Login

Este diagrama detalha o processo de autenticação do administrador.

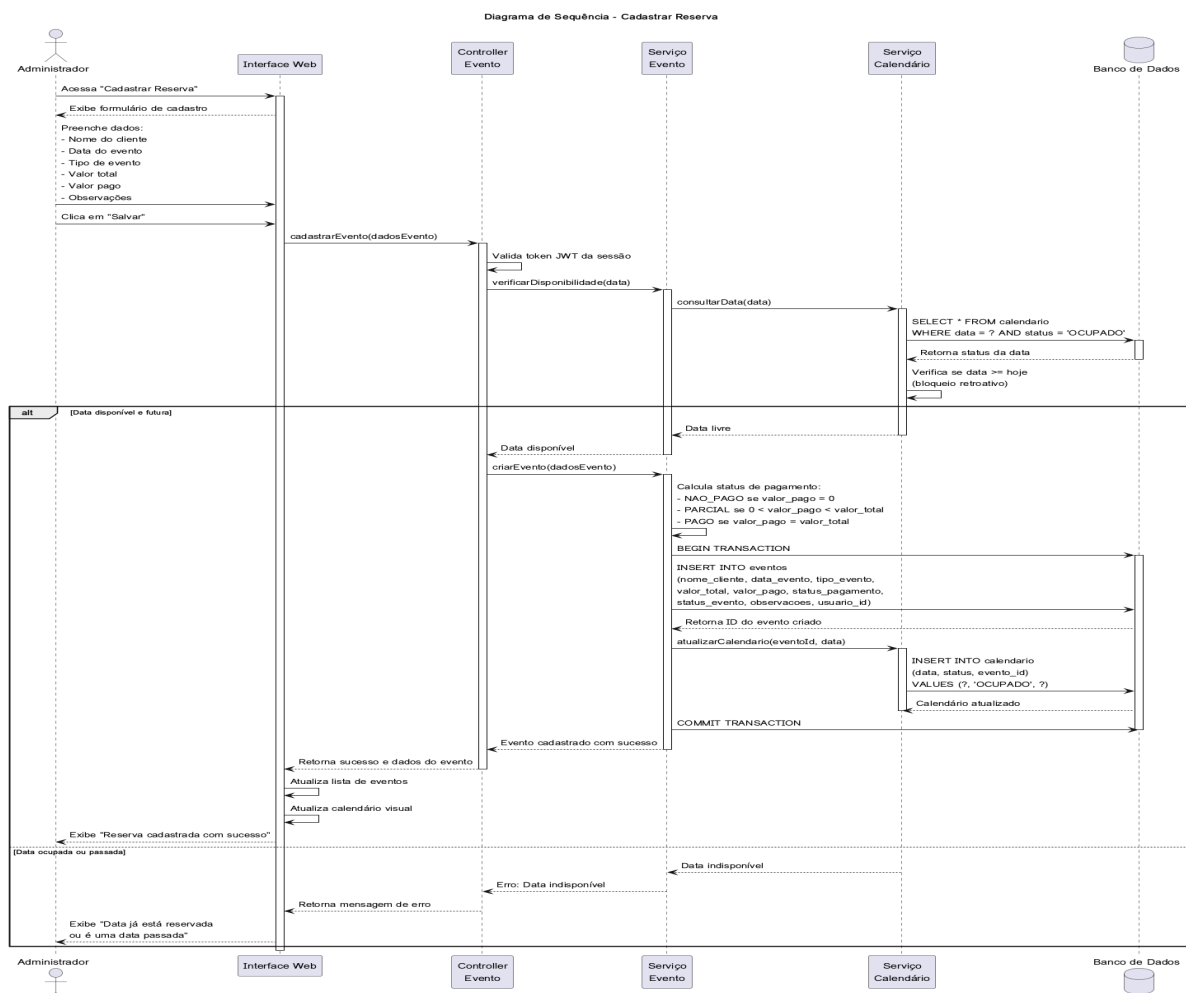


## Descrição do Fluxo:

- 1.O Administrador insere email e senha na Interface Web.
- 2.A interface envia os dados para o Controller de Autenticação.
- 3.O controller aciona o Serviço de Autenticação, que consulta o Banco de Dados para encontrar um usuário com o email e tipo 'ADMIN'.
- 4.O serviço compara o hash da senha fornecida com o hash armazenado no banco usando BCrypt.
- 5.Se as credenciais forem válidas, o sistema gera um token JWT, cria um registro na tabela sessoes e retorna o token para a interface.
- 6.A interface armazena o token no localStorage e redireciona o administrador para o painel de controle.
- 7.Caso contrário, uma mensagem de erro é exibida.

## 3.2. Sequência de Cadastro de Reserva

Este diagrama ilustra o fluxo completo para registrar um novo evento no sistema.



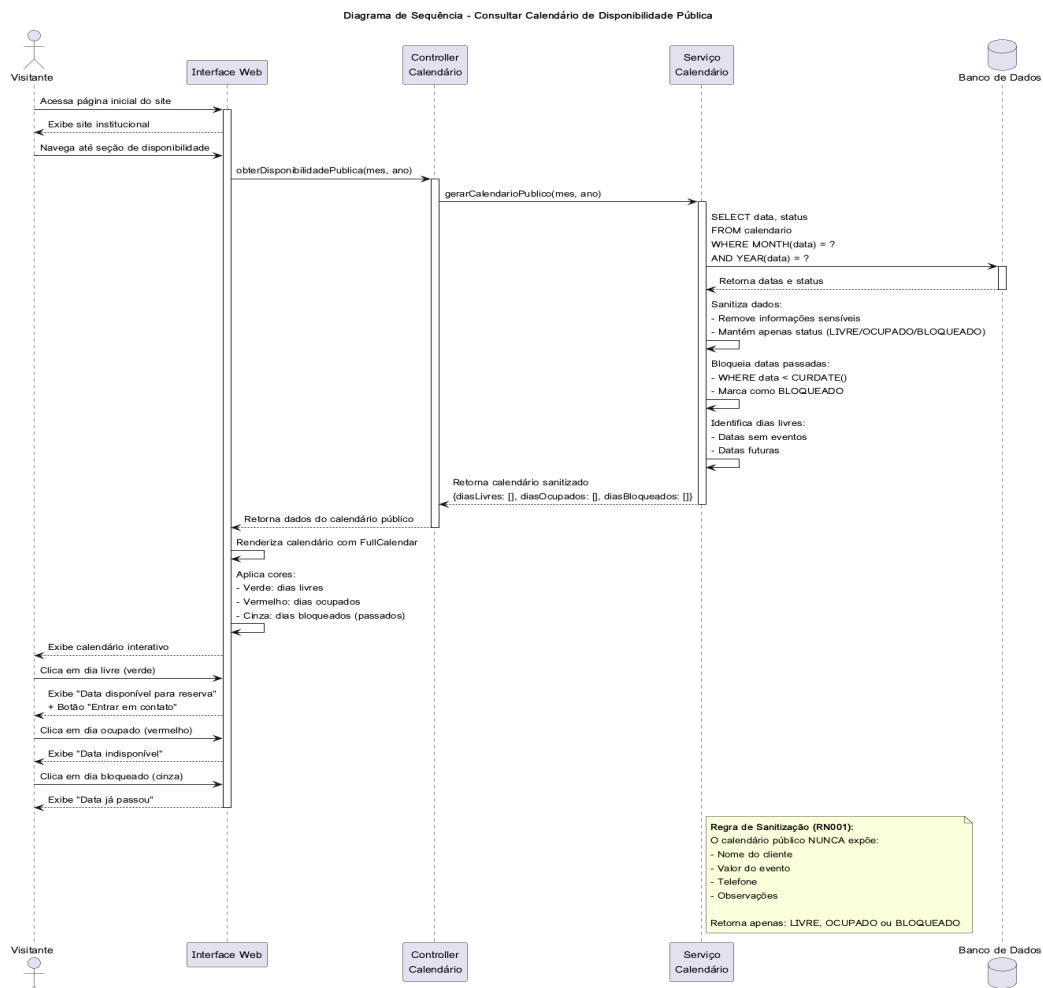


## Descrição do Fluxo:

- 1.O Administrador preenche o formulário de cadastro na Interface Web.
- 2.A interface envia os dados do evento para o Controller de Evento.
- 3.O controller valida a sessão do administrador e aciona o Serviço de Evento para verificar a disponibilidade da data.
- 4.O Serviço de Calendário consulta o banco de dados para garantir que a data está livre e não é uma data passada (RN002).
- 5.Se a data estiver disponível, o sistema inicia uma transação no banco de dados, calcula o status de pagamento, insere o novo registro na tabela eventos e atualiza a tabela calendario, marcando a data como 'OCUPADO'.
- 6.Após o COMMIT da transação, o sistema retorna uma mensagem de sucesso, e a interface atualiza a lista de eventos e o calendário visual.

### 3.3. Sequência de Consulta ao Calendário Público

Este diagrama é crucial, pois mostra como a regra de privacidade da agenda (RN001) é implementada.

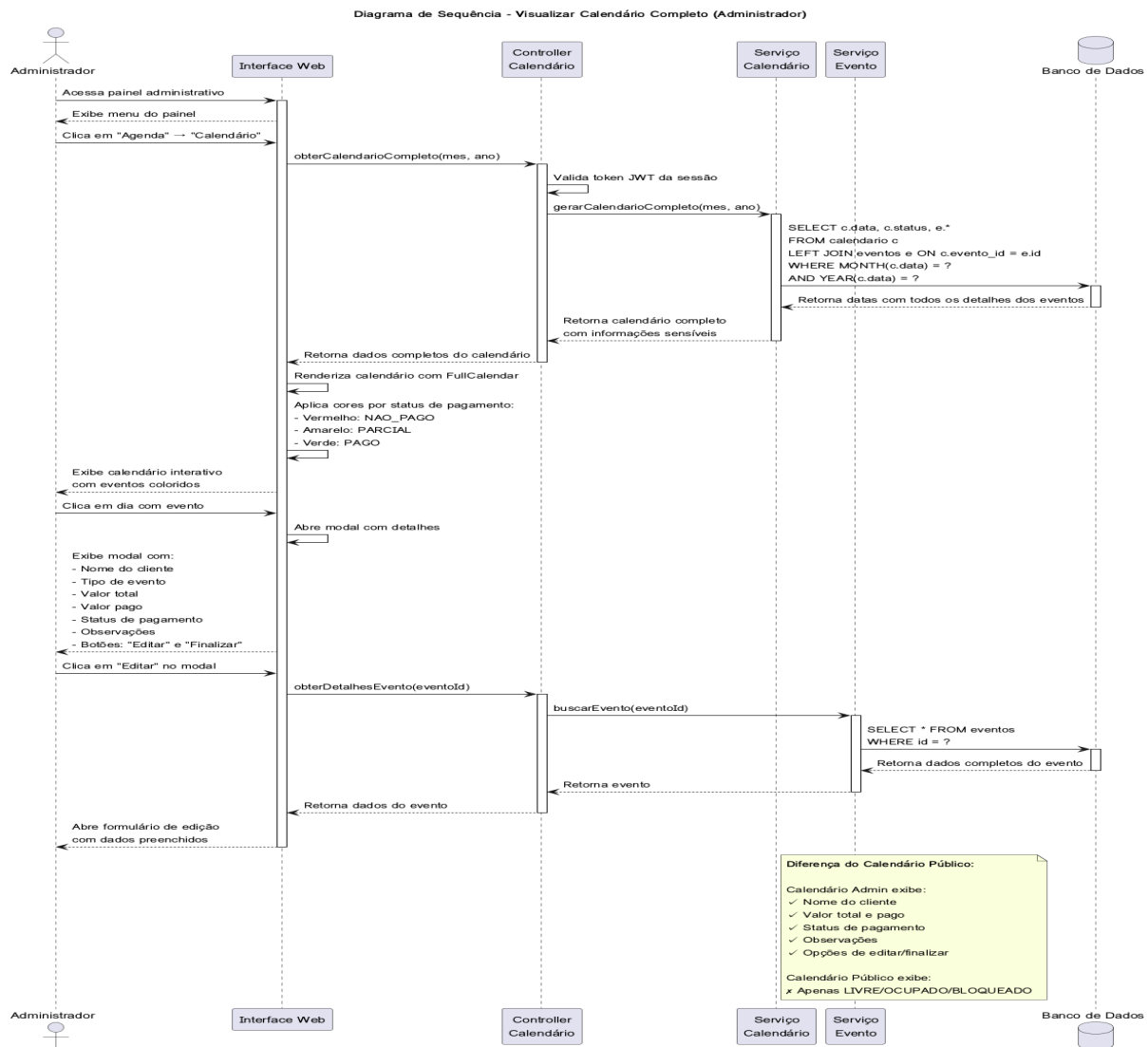


## Descrição do Fluxo:

- 1.O Visitante acessa a seção de disponibilidade no site.
- 2.A Interface Web solicita os dados do calendário público ao Controller de Calendário.
- 3.O Serviço de Calendário consulta o banco de dados e obtém os status das datas (LIVRE, OCUPADO, BLOQUEADO).
- 4.Ponto Crítico: O serviço sanitiza os dados, removendo todas as informações sensíveis (nome do cliente, valores, etc.) e também bloqueia as datas passadas.
- 5.O serviço retorna para a interface apenas um objeto contendo listas de dias livres, ocupados e bloqueados.
- 6.A interface renderiza o calendário com cores, garantindo que o visitante nunca tenha acesso a informações privadas.

## 3.4. Sequência de Visualização do Calendário do Administrador

Este diagrama contrasta com o anterior, mostrando o acesso privilegiado do administrador.

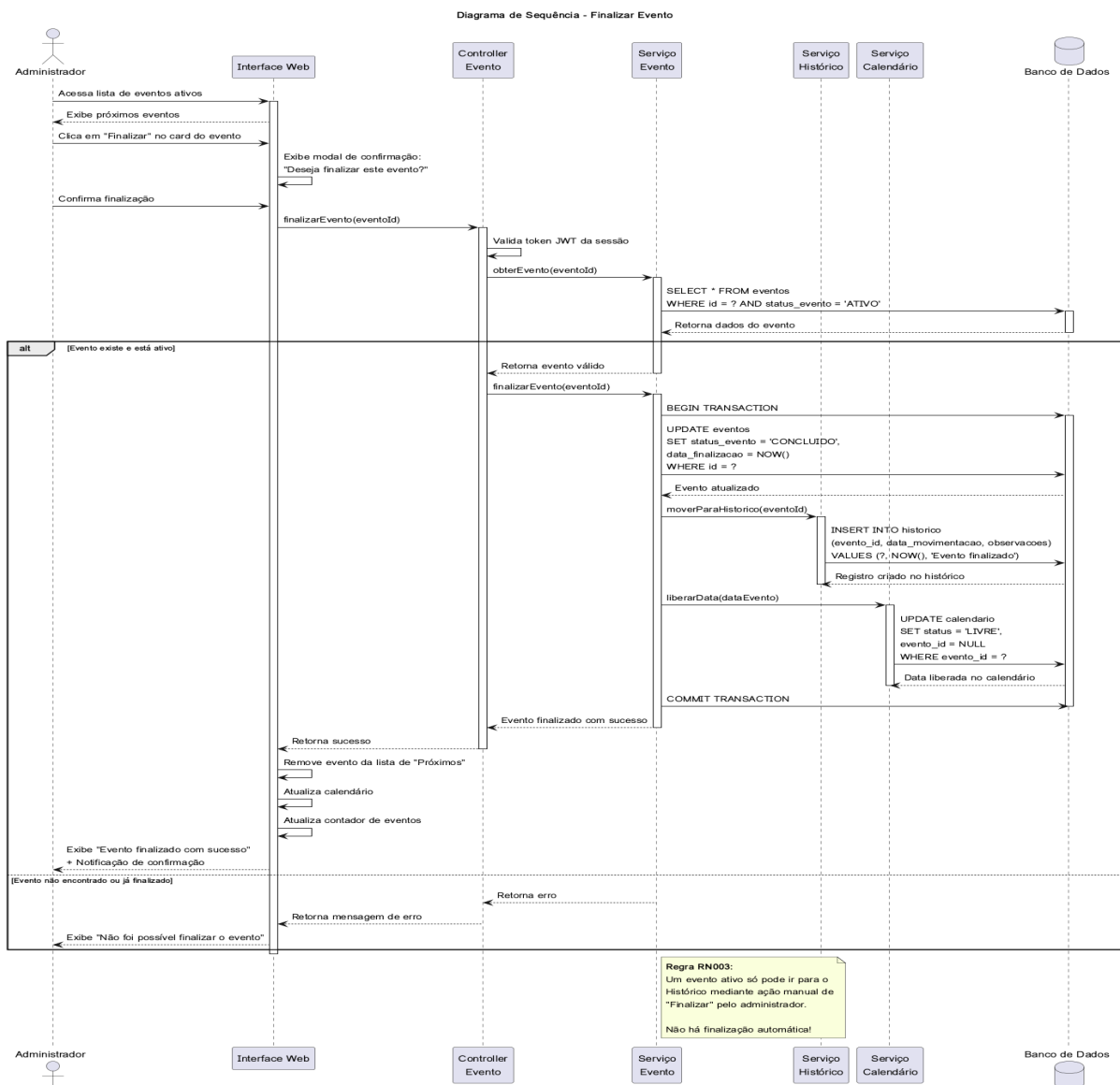


## Descrição do Fluxo:

- 1.O Administrador autenticado acessa a aba "Calendário" no painel.
- 2.A Interface Web solicita os dados do calendário completo ao Controller de Calendário.
- 3.Após validar a sessão, o serviço consulta o banco de dados usando um JOIN entre as tabelas calendario e eventos para obter todos os detalhes dos eventos.
- 4.Os dados completos são retornados para a interface, que renderiza o calendário com cores baseadas no status de pagamento (vermelho, amarelo, verde).
- 5.Ao clicar em um evento, um modal é aberto, exibindo todas as informações sensíveis (nome do cliente, valores, observações) e fornecendo opções para editar ou finalizar o evento.

### 3.5. Sequência de Finalização de Evento

Este diagrama detalha como um evento é movido do estado ativo para o histórico, conforme a regra de negócio RN003.

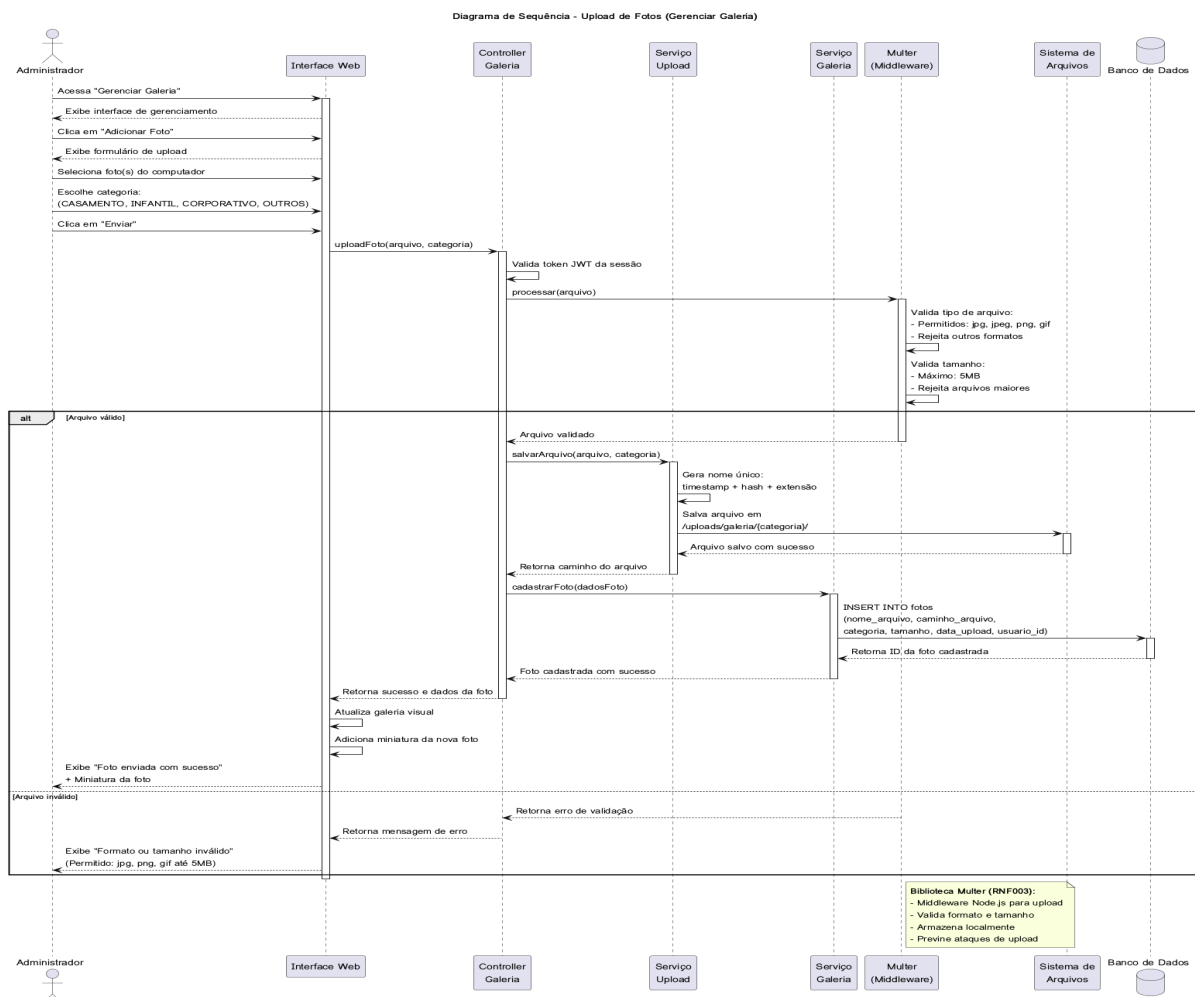


## Descrição do Fluxo:

- 1.O Administrador clica no botão "Finalizar" em um evento na lista de "Próximos Eventos".
- 2.Após a confirmação, o Controller de Evento é acionado.
- 3.O sistema valida a sessão e verifica se o evento existe e está ativo.
- 4.Uma transação é iniciada no banco de dados para garantir a atomicidade da operação.
- 5.O sistema executa três ações: atualiza o status do evento para 'CONCLUIDO', insere um registro na tabela historico e, crucialmente, libera a data no calendário, alterando seu status para 'LIVRE'.
- 6.Após o COMMIT, a interface é atualizada, removendo o evento da lista de ativos.

## 3.6. Sequência de Upload de Fotos

Este diagrama mostra o fluxo de upload de imagens para a galeria, incluindo as validações de segurança.



### **Descrição do Fluxo:**

- 1.O Administrador seleciona um arquivo e uma categoria na interface de gerenciamento da galeria.
- 2.A requisição é enviada para o Controller de Galeria, passando pelo middleware Multer.
- 3.O Multer valida o arquivo, verificando o tipo (MIME type) e o tamanho (máximo de 5MB), conforme o requisito RNF003.
- 4.Se o arquivo for válido, o Serviço de Upload gera um nome de arquivo único para evitar conflitos e salva a imagem no Sistema de Arquivos, em uma pasta correspondente à categoria.
- 5.O Serviço de Galeria insere os metadados da foto (caminho, categoria, etc.) na tabela fotos do banco de dados.
- 6.A interface recebe uma confirmação de sucesso e atualiza a galeria para exibir a nova imagem.

## **9. Registro de Entrevistas e Evolução do Projeto (Iterações)**

Esta seção documenta o processo iterativo de levantamento de requisitos e validação com o cliente (Orientador/Proprietário), detalhando as decisões tomadas em cada etapa do modelo em espiral.

### **Reunião 1: Concepção e Viabilidade**

- Foco: Definição do Escopo e Estratégia de Negócio.
- Pauta: Identificação do nicho de mercado (aluguel de espaço para eventos em Itacoatiara) e o problema da desconfiança dos clientes em negociações informais.
- Decisões Tomadas:
  - a. O sistema deve transmitir profissionalismo para gerar credibilidade (diferencial competitivo na cidade).
  - b. Necessidade de um visual "clean" e organizado.
  - c. Definição de que o sistema teria uma área pública (vitrine) e uma área administrativa.
- Resultado da Iteração: Definição dos Requisitos Iniciais e esboço da identidade visual.

### **Reunião 2: Interface e Regras de Negócio Financeiras**

- Foco: Validação de Prototipação (Front-end) e Fluxo de Pagamento.

- Pauta: Apresentação do layout inicial e discussão sobre como o cliente final visualiza os preços e disponibilidade.
- Decisões Tomadas:
  - a. Aprovação do layout responsivo (Bootstrap).
  - b. Definição de que o pagamento seria via PIX/Transferência, necessitando de confirmação manual no sistema.
  - c. Discussão sobre a funcionalidade de geração de contratos e recibos (deixada como melhoria futura).
  - d. Diferenciação clara entre o que é "Reserva Pendente" e "Reserva Confirmada".
- Resultado da Iteração: Validação da Interface Gráfica e início da codificação do Back-end.

### Reunião 3: Lógica de Dados e Hospedagem

- Foco: Banco de Dados e Refinamento de Lógica.
- Pauta: Correção na ordenação dos eventos e definição de planos de hospedagem.
- Problemas Identificados e Soluções:
  - a. Erro: Os eventos não estavam em ordem cronológica. Solução: Ajuste na *query* SQL e lógica do Back-end para ordenar por data mais próxima.
  - b. Visualização: Implementação do sistema de cores (Semáforo) para status de pagamento (Vermelho: Não pago / Amarelo: Parcial / Verde: Pago).
  - c. Infraestrutura: Decisão de utilizar a *Hostinger* (Plano VPS/Node) para o deploy final, visando performance e profissionalismo.
- Resultado da Iteração: Sistema funcional integrado ao Banco de Dados MySQL.

### Reunião 4: Refinamento Final, Segurança e Deploy

- Foco: Correção de Bugs, Segurança e Funcionalidade "Histórico".
- Pauta: Testes finais de funcionalidades críticas e fluxo de encerramento de eventos.
- Ajustes Realizados:
  - a. Login: Correção de bug onde o login retornava erro de *hash*. Ajuste na biblioteca de criptografia.
  - b. Upload: Implementação de validações no *Multer* para impedir arquivos corrompidos ou nomes vazios.
  - c. Ciclo de Vida: Implementação da lógica "Finalizar Evento", que move o registro da agenda ativa para o histórico e libera a visualização no calendário público, mantendo o registro financeiro.
- Resultado da Iteração: Versão final do software (Release Candidate) pronta para apresentação.

## 10. Estimativa de Esforço e Tempo Investido

Conforme solicitado, abaixo apresentamos o gráfico de tempo investido por funcionalidade/fase do projeto, totalizando o esforço da dupla no desenvolvimento.

| Funcionalidade / Fase       | Atividades Envolvidas                                                                      | Tempo Estimado (Horas) |
|-----------------------------|--------------------------------------------------------------------------------------------|------------------------|
| 1. Levantamento e Análise   | Reuniões, Definição de Requisitos, Diagramas UML (Caso de Uso, Classes).                   | 8h                     |
| 2. Prototipação e Design    | Criação do HTML/CSS, Implementação do Bootstrap 5, Design Responsivo.                      | 12h                    |
| 3. Configuração de Ambiente | Configuração do Node.js, Express, Instalação do MySQL Workbench e Git.                     | 4h                     |
| 4. Desenvolvimento Back-end | Criação de API, Rotas, Controllers, Autenticação (JWT/Bcrypt) e lógica de Upload (Multer). | 20h                    |
| 5. Banco de Dados           | Modelagem (DER), Criação de Tabelas, Queries SQL e Integração.                             | 10h                    |
| 6. Integração Front/Back    | Consumo de APIs via Fetch, Lógica do FullCalendar, Manipulação do DOM.                     | 15h                    |

|                       |                                                                             |            |
|-----------------------|-----------------------------------------------------------------------------|------------|
| 7. Testes e Correções | Debugging (conforme Reuniões 3 e 4), Ajustes de responsividade e Segurança. | 8h         |
| 8. Documentação       | Escrita deste documento, formatação e preparação da apresentação.           | 5h         |
| TOTAL GERAL           |                                                                             | ~ 82 Horas |

*Nota: O tempo inclui o trabalho conjunto e individual da dupla*